

MiniProject 3: Odd-One-Out Image Groups

Aahaan Rawal (aahaanrawal), Ian Gifford (iangifford8), Nathan Hu (nhucanada)

Kaggle Group Name: Group 67

Abstract

We address a novel outlier detection problem where, for each group of 5 grayscale 32x32 images, 4 share a hidden attribute and 1 is an outlier. The task is to identify the outlier's index (5-class classification). We implement a CNN-based model with a shared encoder and MLP comparison head, enforcing a parameter limit of $\leq 25,000$. Our final model achieves a public test accuracy of 80.3% with 18,373 trainable parameters. Key improvements stemmed from optimizing the number of epochs and learning rate schedule rather than increasing model size or adding complexity to the comparison head.

Introduction

The odd-one-out problem presents a unique challenge: the defining attribute shared by the majority of images is unknown and may vary across examples. This prevents the model from relying on fixed semantic features and instead requires it to learn relative comparisons within each group. Additionally, the dataset is relatively small (3000 examples), which increases the risk of overfitting and limits the effectiveness of large models. To address these challenges, we design a model that explicitly encodes relational reasoning. Each image is mapped into a learning embedding space using a shared CNN encoder. Instead of directly classifying each image independently, the model evaluates how much each embedding deviates from the statistical structure of the remaining images. This approach aligns naturally with the structure of the problem and enables efficient learning within the strict parameter budget.

Data Processing

The dataset consists of 3000 training examples, each containing 5 grayscale images of size 32x32. Labels indicate the index of the outlier image. To prepare the data for training, pixel values were first normalized to the range $[0, 1]$ by dividing by 255, followed by standardization using the mean and standard deviation computed over the dataset. This ensured that the inputs were centered and scaled appropriately, which is particularly important for stable neural network training. Additionally, a channel dimension was introduced to make the data compatible with convolutional neural networks, transforming each sample from shape (5, 32, 32) to (5, 1, 32, 32).

To improve generalization, several data augmentation techniques were applied dynamically during training. These included rotations by multiples of 90 degrees, horizontal and vertical reflections, and translations of up to two pixels in both spatial directions. Importantly, these transformations were applied consistently across all five images within a group to preserve their relative structure. Gaussian noise with a small standard deviation (0.05) was also added to encourage robustness to minor pixel-level variations. In addition to augmentation, a permutation strategy was used in which the order of the five images in each group was randomly shuffled during training, with the corresponding label updated accordingly. This prevented the model from learning positional biases and ensured that predictions were based solely on image content.

Initially, a stratified train-validation split was used to monitor performance and guide hyperparameter tuning. However, subsequent experiments revealed that training on the full dataset for a larger number of epochs resulted in improved performance, suggesting that maximizing the use of available data was more beneficial than reserving a portion for validation.

Methods

The proposed model consists of two main components: a shared convolutional encoder and a comparison-based multilayer perceptron (MLP) head, designed to explicitly capture relational differences

between images within each group. Each of the five input images is processed independently through the shared encoder, ensuring consistent feature extraction while maintaining a low parameter count. The encoder is composed of four convolutional layers, each followed by Group Normalization and GELU activation functions. These layers progressively reduce the spatial resolution while increasing the depth of the feature maps, enabling the network to learn increasingly abstract representations. An adaptive average pooling layer reduces the spatial dimensions to a fixed size, and the resulting feature maps are flattened and passed through a linear projection layer to produce a 24-dimensional embedding vector for each image. See Figure 2 in the Appendix for a diagram of the full model.

To determine which image is the outlier, the model compares each embedding to the distribution of the remaining embeddings in the group. For a given image embedding e_i , the embeddings of the remaining four images are aggregated by computing their mean and variance. A feature vector is then constructed by concatenating the embedding itself, the mean of the other embeddings, the absolute difference between the embedding and this mean, and the variance of the remaining embeddings. This representation captures both how far the image lies from the central tendency of the group and how dispersed the group is, allowing the model to quantify the degree of deviation. These features are passed through a small MLP consisting of a single hidden layer with dimension 40 and a final linear layer that outputs a scalar score. This process is repeated for each of the five images, producing five scores that are treated as logits and passed into a cross-entropy loss function.

The model was trained using the Adam optimizer with a learning rate of 0.005 and weight decay of $1e-4$. Label smoothing with a factor of 0.02 was used to improve generalization. A cosine annealing learning rate scheduler was employed, with the parameter T_{max} carefully aligned with the total number of training epochs (100) to ensure a smooth decay in learning rate. This alignment was found to be important, as smaller values of T_{max} led to suboptimal learning rate behavior and reduced performance.

Hyperparameters were selected using 3-fold stratified cross-validation over a predefined search space, evaluating each configuration based on mean validation loss and accuracy across folds. The final configuration was chosen as the one achieving the lowest mean validation loss, with a corresponding mean validation loss of 0.4118 and mean validation accuracy of 88.87%. The selected hyperparameters were a batch size of 32, embedding dimension of 24, hidden dimension of 40, learning rate of 0.005, weight decay of $1e-4$, label smooth of 0.02, and a cosine annealing scheduler over 100 epochs. This systematic selection process ensured that the chosen configuration generalized well across different data splits rather than overfitting into a single validation set. See Table 1 in the Appendix for the top ten results from cross validation.

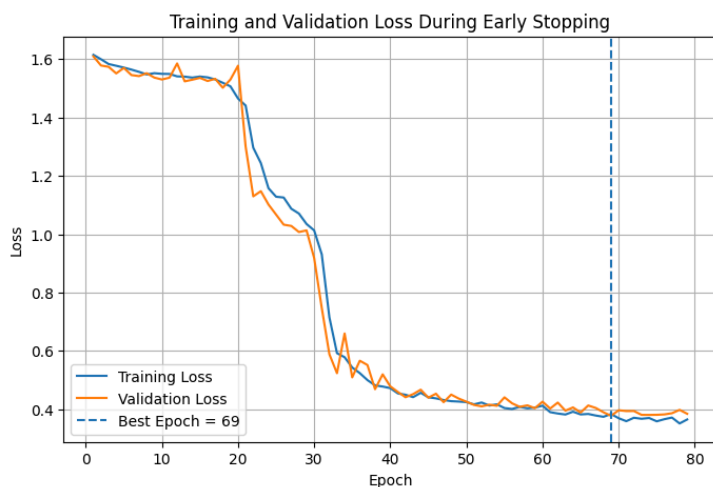
The model architecture was explicitly designed to satisfy the constraint of having at most 25,000 trainable parameters. The total number of parameters was computed programmatically in PyTorch by summing all parameters with `requires_grad=True`, resulting in 18,373 trainable parameters. During development, alternative architectures with increased encoder complexity were explored, reaching up to approximately 23,000 parameters; however, these larger models did not yield improvements in validation or test performance. This indicates that the final architecture is not limited by capacity, but rather is well-matched to the complexity of the task.

In addition to standard inference, test-time augmentation (TTA) was employed to improve prediction robustness. Multiple transformed versions of each input group were evaluated at test time, including rotations, reflections, and random permutations of image order. The model’s predictions were averaged across these augmented inputs to produce the final output. This approach reduces variance and improves generalization without increasing the number of trainable parameters. Empirically, removing test-time augmentation resulted in an approximate 3% decrease in accuracy, demonstrating its effectiveness as a simple yet impactful enhancement.

Results and Analysis

The performance of the proposed model was evaluated using both internal validation metrics and the public test set provided through the Kaggle competition. During development, a stratified 80/20 train-validation split was used to monitor generalization and guide model selection. The best validation accuracy achieved during early stopping was 91.83%, indicating that the model is highly effective at capturing the underlying structure that distinguishes the outlier image within each group. In addition, 3-fold cross-validation was performed during hyperparameter tuning, where the selected configuration achieved a mean validation accuracy of 88.87% and a mean validation loss of 0.4118, demonstrating stable and consistent performance across different data splits.

Figure 1: Training and Validation Loss Plot



The training dynamics of the model are illustrated through the learning curve (training and validation loss over epochs). As shown in Figure 1, the training and validation losses decrease steadily during training, with a noticeable transition from underfitting to effective learning around epoch 20–30.

From the graph, it is evident that the model initially struggles to learn meaningful patterns, as both training and validation losses remain high and relatively flat during the first 15–20 epochs. However, a sharp decrease in both losses occurs shortly afterward, corresponding to a rapid improvement in validation accuracy. This phase indicates that the model begins to successfully extract discriminative features

for identifying the outlier. Beyond this point, both curves continue to decrease more gradually, with the validation loss stabilizing and reaching its minimum around epoch 70–80. The relatively small gap between training and validation loss throughout training suggests that overfitting is well-controlled, likely due to the use of data augmentation, label smoothing, and weight decay.

To further investigate training behavior, additional experiments were conducted with different epoch settings. When training was extended to 150 epochs, the training accuracy continued to increase substantially, while the validation and test accuracy remained approximately constant at around 80%. This divergence between training and test performance provides strong evidence of overfitting, indicating that additional training beyond 100 epochs does not improve generalization. As a result, the choice of 100 epochs represents a well-justified balance between sufficient learning and avoiding overfitting.

The final model was trained on the full training dataset using the selected hyperparameters and evaluated on the public portion of the test set. The resulting public test accuracy was 80.30%, significantly outperforming the logistic regression baseline, which performs near random chance (20%). This demonstrates the effectiveness of the learned feature representations and the comparison-based architecture.

Several ablation experiments were conducted to evaluate the contribution of different design choices. First, replacing the GELU activation function with ReLU resulted in a noticeable decrease in performance, suggesting that GELU provides smoother nonlinearities that are better suited for this task. Second, a more complex comparison mechanism was explored, in which pairwise differences between

embeddings were explicitly computed and passed through the MLP. Despite increasing model complexity, this approach led to lower test accuracy, indicating that the simpler statistical comparison using mean and variance is both more efficient and more effective.

Additionally, increasing the complexity of the CNN encoder to approach the parameter limit (approximately 23,000 parameters) did not yield any improvement in performance, suggesting that the task does not benefit from increased representational capacity beyond a certain point. This reinforces the idea that the key challenge lies in relational reasoning rather than feature extraction alone.

Finally, the impact of test-time augmentation (TTA) was evaluated by comparing performance with and without its use. Removing TTA resulted in an average decrease in accuracy of approximately 3%, confirming that aggregating predictions across transformed inputs provides a meaningful boost in performance. This highlights the importance of robustness to spatial transformations and input ordering in this task.

Discussion and Conclusion

The results demonstrate that the success of the proposed model is primarily driven by its ability to explicitly model relationships between images within each group, rather than relying solely on individual feature extraction. The use of a shared encoder ensures that all images are represented in a consistent embedding space, while the comparison-based feature construction allows the model to directly measure how much each image deviates from the others. This inductive bias is well-aligned with the structure of the task and appears to be a key factor in achieving strong performance.

Several design choices contributed positively to the model’s effectiveness. Data augmentation played a crucial role in improving generalization by exposing the model to a wider range of transformations while preserving the relative structure between images. Label smoothing further helped prevent overconfident predictions, leading to more stable training. The cosine annealing learning rate scheduler was also important, particularly when its cycle length was aligned with the total number of training epochs, as this ensured a smooth and consistent optimization trajectory. Additionally, test-time augmentation provided a significant improvement in final accuracy by reducing prediction variance.

On the other hand, certain modifications were found to be ineffective or even detrimental. Increasing model complexity, either through a larger CNN encoder or a more sophisticated comparison mechanism, did not improve performance and in some cases reduced accuracy. This suggests that the model is not capacity-limited, but rather that the chosen architecture already captures the essential structure of the problem. Similarly, extending training beyond 100 epochs led to clear overfitting, as evidenced by increasing training accuracy without corresponding gains in test performance. These findings emphasize the importance of balancing model capacity and training duration.

Despite its strong performance, the proposed approach has several important limitations. First, the comparison mechanism relies on relatively simple statistical summaries, specifically, the mean and variance of the embeddings of the other images. While effective, these statistics only capture first- and second-order information and may fail to represent more complex relationships or multimodal structures within the group. As a result, the model may struggle in cases where the “normal” images are themselves diverse or where the outlier differs in a subtle, higher-order way that is not reflected in these statistics. Additionally, the model treats each comparison independently and does not explicitly model interactions between all pairs of images simultaneously, which could limit its ability to capture richer relational patterns.

Another limitation lies in the reliance on test-time augmentation to achieve optimal performance. While TTA improves accuracy, it increases computational cost at inference time, as multiple forward passes are

required for each input. This may not be practical in real-time or resource-constrained settings. Furthermore, although permutation augmentation helps enforce permutation invariance during training, the model architecture itself is not inherently permutation-invariant, meaning it still relies on data augmentation rather than structural guarantees. Finally, the relatively small size of the dataset (3,000 training examples) may limit the model's ability to generalize to more complex or diverse scenarios, and may partially explain the gap between validation and test performance.

There are several promising directions for future work that could address these limitations and further improve performance. One natural extension is to replace the fixed statistical comparison mechanism with a learnable aggregation function. For example, attention-based models could be used to dynamically weight the importance of each image in the group when computing comparisons, allowing the model to capture more nuanced and context-dependent relationships. Transformer-style architectures, which have been highly successful in modeling interactions between sets of elements, could provide a powerful framework for this task by enabling full pairwise interactions between all images.

Another potential direction is the use of permutation-invariant architectures, such as Deep Sets or set transformers, which would allow the model to inherently respect the unordered nature of the input without relying on permutation-based augmentation. This could improve both efficiency and generalization by embedding permutation invariance directly into the model structure. Additionally, graph-based approaches could be explored, where each image is treated as a node and edges represent relationships between images, enabling the use of graph neural networks to model higher-order interactions.

Further improvements could also be achieved by enhancing the encoder component. While increasing its size did not yield improvements in this setting, alternative architectures, such as residual connections or lightweight attention modules, could improve feature extraction without significantly increasing parameter count. Another avenue is to incorporate contrastive or self-supervised pretraining, which could help the model learn more robust and transferable representations before fine-tuning on the outlier detection task.

Finally, future work could aim to reduce or eliminate the need for test-time augmentation by incorporating its benefits directly into training. Techniques such as consistency regularization or training with stronger augmentations could encourage the model to produce stable predictions across transformations, thereby improving robustness without increasing inference cost.

In conclusion, this work demonstrates that a carefully designed, parameter-efficient model with explicit relational reasoning can achieve strong performance on the odd-one-out task. By combining a lightweight CNN encoder with a structured comparison mechanism and effective training strategies, the model achieves a public test accuracy of 80.30% while remaining well within the parameter constraint. The results highlight that performance gains in this task are driven more by how relationships are modeled than by increasing model size, and suggest that future improvements will likely come from more expressive and principled approaches to relational learning.

Statement of Contributions (1–3 sentences)

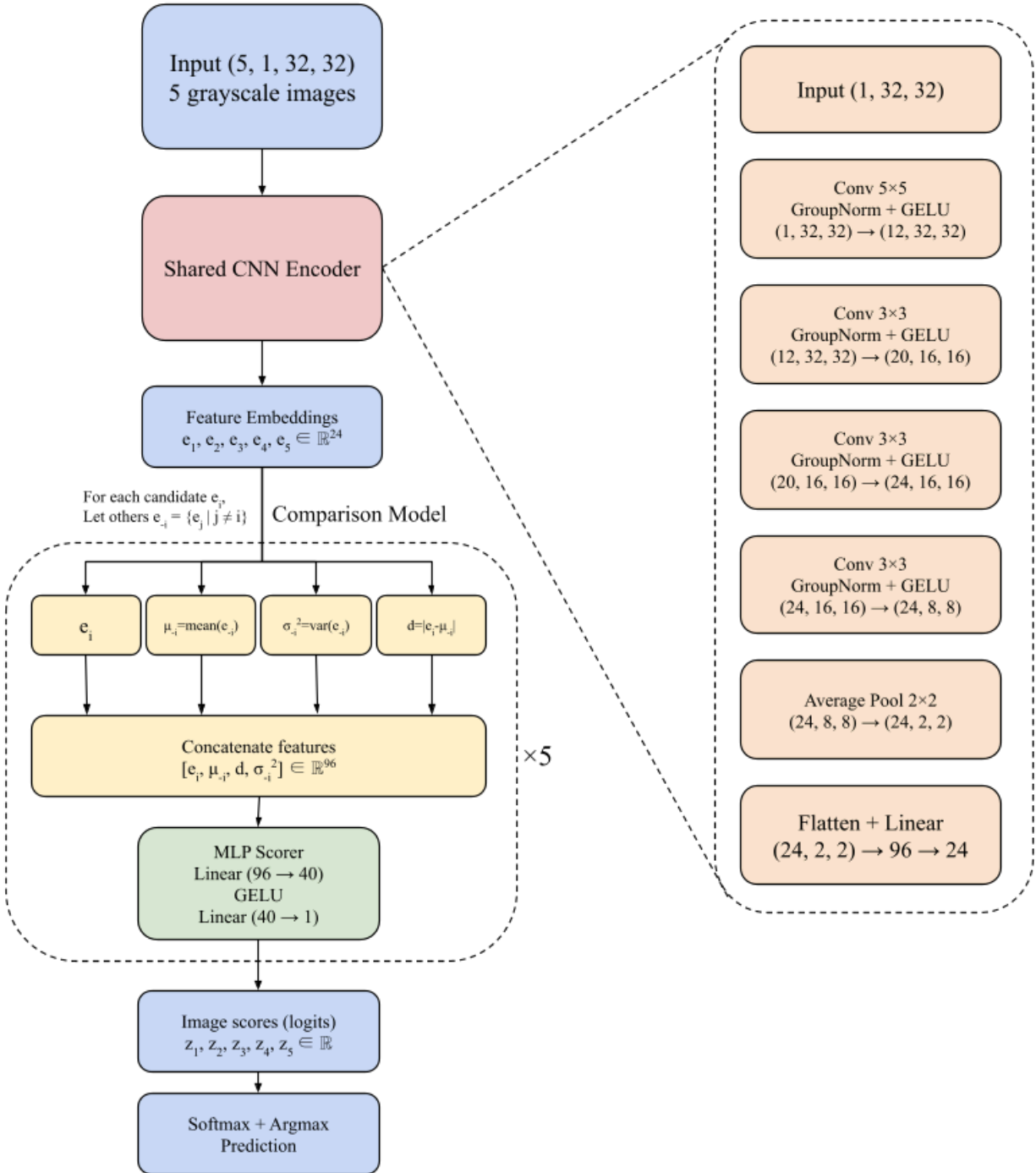
Ian handled most of the code by choosing the model architecture and performing hyperparameter selection experiments, while Aahaan handled most of the writeup. In the end, we collaboratively proofread each other's work and made sure each part was consistent. Unfortunately, Nathan was injured leading up to the deadline of the assignment and could not contribute as much as planned, although he assisted in proofreading our notebook and writeup.

Statement on the use of LLMs and other resources (1–3 sentences)

LLMs were used to help us justify certain decisions, and to assist development whenever we were stuck and had invested significant time with little success. This looked like assistance in planning development by providing feedback and clarification on how to appropriately apply course concepts, and was not used in implementing solutions or writing code except as a form of documentation.

Appendix

Figure 2: Odd-one-out Model Architecture



Each of the five images is processed by a shared CNN encoder to produce a fixed-dimensional embedding. For each candidate image, the model computes statistics over the remaining four embeddings (mean and variance). These features are concatenated and passed through a small MLP to produce a scalar score. The model outputs five logits corresponding to the likelihood of each image being the odd one out.

Table 1: Top 10 results of cross validations sorted by validation loss

#	Batch Size	LR	WD	Val Loss	Std	Val Acc	Std
0	32	0.005	1.00E-05	0.419	0.039	0.889	0.011
1	32	0.005	1.00E-04	0.420	0.035	0.890	0.009
2	32	0.001	1.00E-04	0.440	0.020	0.888	0.004
3	32	0.001	1.00E-05	0.448	0.042	0.879	0.010
4	32	0.0007	1.00E-04	0.602	0.096	0.855	0.018
5	32	0.0007	1.00E-05	0.733	0.157	0.803	0.063
6	64	0.001	1.00E-04	0.776	0.045	0.798	0.019
7	128	0.005	1.00E-05	0.780	0.511	0.707	0.272
8	64	0.005	1.00E-04	0.787	0.503	0.692	0.280
9	64	0.005	1.00E-05	0.792	0.513	0.693	0.271

Validation performance across different hyperparameter configurations (see notebook for exact search space). Results report mean and standard deviation over runs of 3-fold cross validation. The best-performing configuration achieves a validation loss of 0.419 with low variance.